

Developer Habit & Productivity Tracker: Complete Build Guide

A Step-by-Step Implementation Plan using Bootify, Spring Boot, MyBatis, and Visualization

Introduction

This guide walks you through building the **Developer Habit & Productivity Tracker** — a web application where you log your daily coding sessions, track the technologies you practice, and visualize your progress through a GitHub-style contribution calendar, streak counters, and skill distribution charts.

The application is built entirely on the stack you selected: **Bootify** for scaffolding, **Spring Boot** for the backend, **MySQL** as the database, **MyBatis** for analytics queries, and **Chart.js + Cal-Heatmap** for visualization. Deployment targets **Railway**, which offers the best developer experience for this stack at minimal cost.

What You Will Build

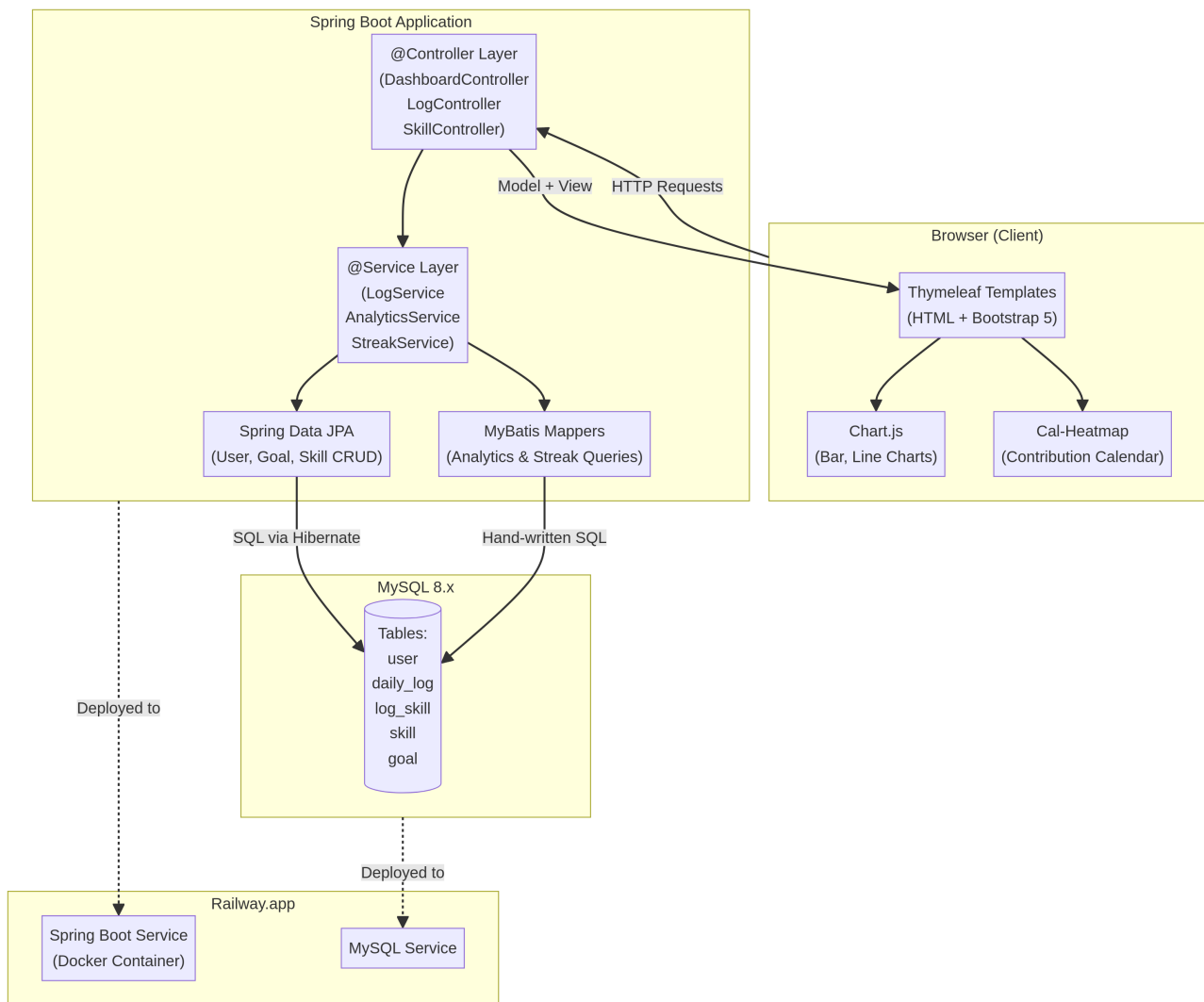
By the end of this guide, you will have a fully deployed web application with the following capabilities:

Feature	Description
Daily Log Entry	A form to record coding hours, a focus score (1–10), and notes for each day.
Skill Tagging	Attach one or more skills (e.g., Java, Spring Boot, React) to each log entry, with minutes practiced.
Contribution Heatmap	A GitHub-style calendar showing your coding activity intensity over the past 12 months.
Streak Counter	Calculates and displays your current consecutive-day coding streak.
Skills Pie Chart	A breakdown of which technologies you’ve spent the most time on in the last 30 days.
Weekly Hours Bar Chart	A bar chart showing your total coding hours per week for the last 12 weeks.
Goal Tracking (MVP)	A simple table showing your active goals and current progress.

Part 1: Architecture & Data Model

Application Architecture

The application follows a classic three-tier architecture: a server-side rendered frontend using Thymeleaf, a Spring Boot backend with a clear service layer, and a MySQL database. We use **two data access strategies in parallel**: Spring Data JPA handles simple CRUD operations on entities (which Bootify generates for us), while **MyBatis handles all complex analytical queries** that require hand-tuned SQL — such as streak calculations, skill aggregations, and time-series data for charts.



Database Schema

The schema is designed around the `daily_log` table as the central entity. Each log entry belongs to a user and can be linked to multiple skills via the `log_skill` join table. This normalized design allows flexible querying and easy future extension.

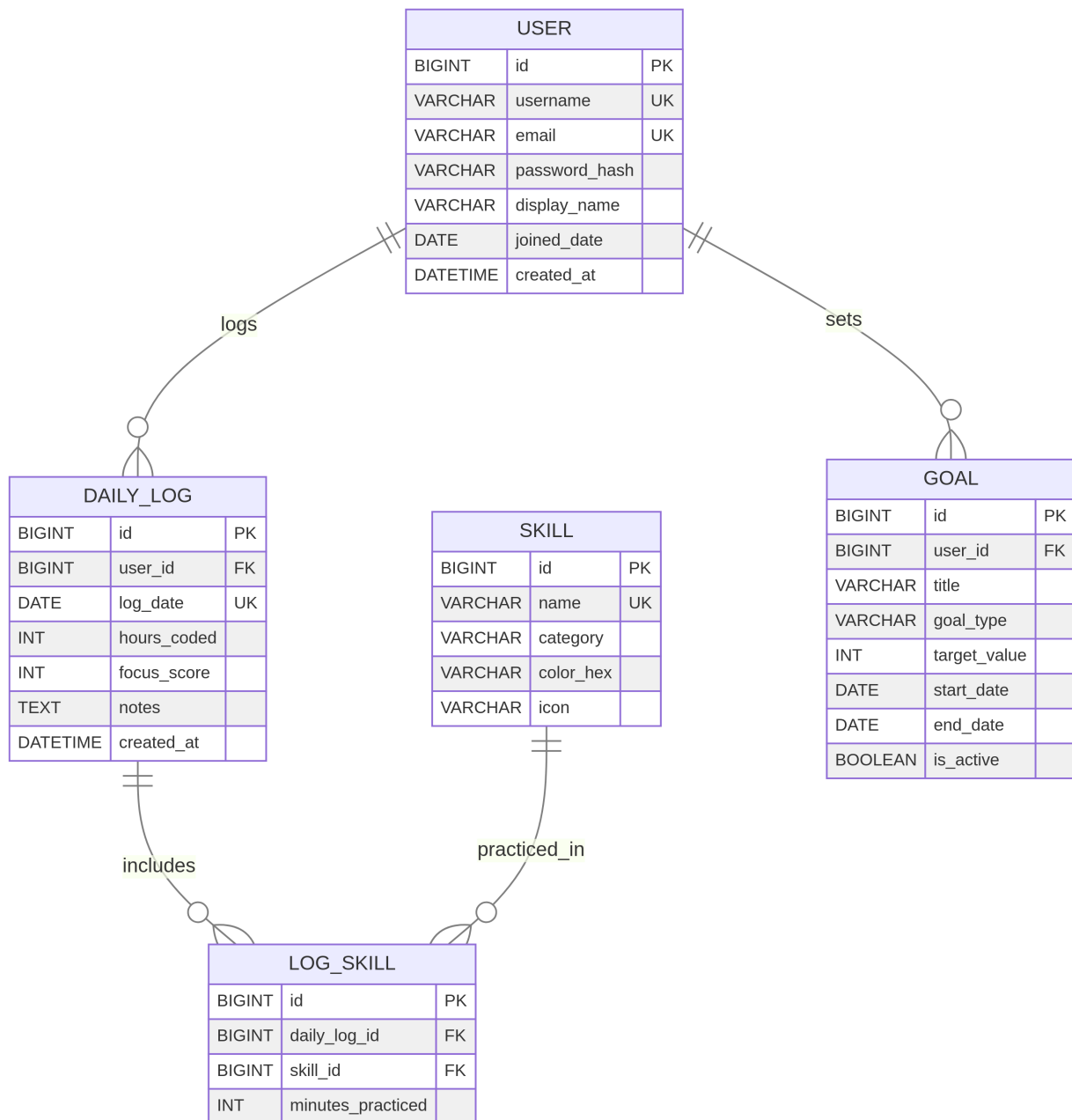


Table Descriptions:

Table	Purpose
user	Stores user credentials and profile information.
skill	A reference table of technologies (e.g., Java, Python, Docker).
daily_log	One row per user per day; records hours coded and focus score.
log_skill	Links a daily log to the skills practiced, storing minutes per skill.
goal	Stores user-defined targets (e.g., “Code 5 hours/week for 4 weeks”).

Part 2: Project Scaffolding with Bootify (Day 1)

Step 2.1 — Generate the Project

Navigate to bootify.io and click **Start Project** (no registration required on the free plan).

General Tab — Project Settings:

Configure the following values in the **General** tab:

Setting	Value
Project Group	com.yourname
Project Name	dev-habit-tracker
Build Type	Maven
Language	Java
Java Version	21
Database	MySQL
Frontend Stack	Thymeleaf
CSS Framework	Bootstrap
Enable Lombok	Yes
Enable Swagger UI	Yes (useful for testing APIs)

Entities Tab — Define the Data Model:

Create the following entities. For each entity, click **Add Entity**, then add the fields listed.

Entity 1: Skill

Field Name	Type	Notes
name	String	Required, Unique
category	String	Optional (e.g., “Language”, “Framework”)
colorHex	String	Optional (e.g., #4A90D9)

Entity 2: DailyLog

Field Name	Type	Notes
logDate	LocalDate	Required, Unique per user
hoursCoded	Integer	Required
focusScore	Integer	Optional (1–10)
notes	String (Text)	Optional

Entity 3: LogSkill

Field Name	Type	Notes
minutesPracticed	Integer	Required

Relationships:

After creating the entities, define the following relationships in the Entities tab:

- `DailyLog` → **Many-to-One** → `User` (field name: `user`)
- `LogSkill` → **Many-to-One** → `DailyLog` (field name: `dailyLog`)
- `LogSkill` → **Many-to-One** → `Skill` (field name: `skill`)

Enable **REST API** generation for `Skill`, `DailyLog`, and `LogSkill`. Then click **Download** to get the project zip.

Step 2.2 — Initialize the Local Environment

Unzip the project, open it in your IDE (IntelliJ IDEA recommended), and perform the following setup steps.

Create the local MySQL database:

```
CREATE DATABASE dev_habits CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'habituser'@'localhost' IDENTIFIED BY 'habitpass';
GRANT ALL PRIVILEGES ON dev_habits.* TO 'habituser'@'localhost';
FLUSH PRIVILEGES;
```

Update `src/main/resources/application.properties` :

```
spring.datasource.url=jdbc:mysql://localhost:3306/dev_habits?
useSSL=false&serverTimezone=UTC
spring.datasource.username=habituser
spring.datasource.password=habitpass
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
```

Initialize Git:

```
cd dev-habit-tracker
git init
git add .
git commit -m "feat: initial scaffold from Bootify"
```

Verify the application starts:

```
./mvnw spring-boot:run
```

Open `http://localhost:8080` in your browser. You should see the Thymeleaf home page and can verify the Swagger UI at `http://localhost:8080/swagger-ui.html`.

Part 3: MyBatis Integration for Analytics (Day 2)

Spring Data JPA (generated by Bootify) handles all standard CRUD operations. MyBatis is added specifically for the complex analytical queries that power the dashboard.

Step 3.1 — Add the MyBatis Dependency

Add the following to the `<dependencies>` section of your `pom.xml` :


```
<dependency>
  <groupId>org.mybatis.spring.boot</groupId>
  <artifactId>mybatis-spring-boot-starter</artifactId>
  <version>3.0.3</version>
</dependency>
```

Step 3.2 — Configure MyBatis

Add the following lines to `application.properties`:

```
# MyBatis
mybatis.mapper-locations=classpath:mapper/*.xml
mybatis.configuration.map-underscore-to-camel-case=true
mybatis.configuration.default-fetch-size=100
mybatis.configuration.default-statement-timeout=30
```

Step 3.3 — Create Model Classes for Query Results

Create a `model` package under your main source directory and add the following classes. These are simple POJOs (Plain Old Java Objects) that MyBatis will map query results into.

`src/main/java/com/yourname/tracker/model/ActivityPoint.java`:

```
package com.yourname.tracker.model;

import lombok.Data;

@Data
public class ActivityPoint {
    private Long timestamp;    // Unix timestamp of the log_date
    private Integer value;    // hours_coded
}
```

`src/main/java/com/yourname/tracker/model/SkillStat.java`:

```
package com.yourname.tracker.model;

import lombok.Data;

@Data
public class SkillStat {
    private String skillName;
    private String color;
    private Long totalMinutes;
}
```

src/main/java/com/yourname/tracker/model/WeeklyHours.java :

```
package com.yourname.tracker.model;

import lombok.Data;

@Data
public class WeeklyHours {
    private String weekLabel;    // e.g., "2025-W04"
    private Integer totalHours;
}
```

Step 3.4 — Create the Mapper Interface

Create `src/main/java/com/yourname/tracker/mapper/AnalyticsMapper.java` :

```
package com.yourname.tracker.mapper;

import com.yourname.tracker.model.ActivityPoint;
import com.yourname.tracker.model.SkillStat;
import com.yourname.tracker.model.WeeklyHours;
import org.apache.ibatis.annotations.Mapper;
import org.apache.ibatis.annotations.Param;

import java.time.LocalDate;
import java.util.List;

@Mapper
public interface AnalyticsMapper {

    /** Returns daily activity data for the Cal-Heatmap. */
    List<ActivityPoint> getDailyActivity(
        @Param("userId") Long userId,
        @Param("startDate") LocalDate startDate,
        @Param("endDate") LocalDate endDate
    );

    /** Returns the current consecutive-day streak for the user. */
    Integer getCurrentStreak(
        @Param("userId") Long userId,
        @Param("today") LocalDate today
    );

    /** Returns skill time distribution for the last N days. */
    List<SkillStat> getSkillDistribution(
        @Param("userId") Long userId,
        @Param("sinceDate") LocalDate sinceDate
    );

    /** Returns weekly total hours for the last 12 weeks. */
    List<WeeklyHours> getWeeklyHours(
        @Param("userId") Long userId,
        @Param("sinceDate") LocalDate sinceDate
    );

    /** Returns the total number of logs for a user. */
    Integer getTotalDaysLogged(@Param("userId") Long userId);
}
```

Step 3.5 — Write the MyBatis XML Mapper

Create the directory `src/main/resources/mapper/` and add the file `AnalyticsMapper.xml`. This is the heart of the analytics layer.

```

<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="com.yourname.tracker.mapper.AnalyticsMapper">

    <!-- =====
        Cal-Heatmap Data: returns Unix timestamp + hours
        ===== -->
    <select id="getDailyActivity"
resultType="com.yourname.tracker.model.ActivityPoint">
        SELECT
            UNIX_TIMESTAMP(d1.log_date) AS timestamp,
            d1.hours_coded                AS value
        FROM daily_log d1
        WHERE d1.user_id = #{userId}
            AND d1.log_date BETWEEN #{startDate} AND #{endDate}
        ORDER BY d1.log_date ASC
    </select>

    <!-- =====
        Current Streak: consecutive days from today backwards
        Uses a recursive CTE to walk back day by day.
        ===== -->
    <select id="getCurrentStreak" resultType="java.lang.Integer">
        WITH RECURSIVE streak_walk (check_date, day_num) AS (
            -- Anchor: start from today
            SELECT #{today}, 1
            UNION ALL
            -- Recursive: go one day back
            SELECT DATE_SUB(check_date, INTERVAL 1 DAY), day_num + 1
            FROM streak_walk
            WHERE day_num < 365
        )
        SELECT COALESCE(COUNT(*), 0)
        FROM (
            SELECT sw.check_date
            FROM streak_walk sw
            WHERE EXISTS (
                SELECT 1 FROM daily_log d1
                WHERE d1.user_id = #{userId}
                    AND d1.log_date = sw.check_date
            )
        )
        -- Stop at the first gap
    </select>

```

```

        AND sw.day_num = (
            SELECT MIN(s2.day_num) FROM streak_walk s2
            WHERE s2.day_num <= sw.day_num
            AND NOT EXISTS (
                SELECT 1 FROM daily_log dl2
                WHERE dl2.user_id = #{userId}
                AND dl2.log_date = s2.check_date
            )
        ) - 1
    OR sw.day_num <= (
        SELECT COALESCE(MIN(s3.day_num), 365)
        FROM streak_walk s3
        WHERE NOT EXISTS (
            SELECT 1 FROM daily_log dl3
            WHERE dl3.user_id = #{userId}
            AND dl3.log_date = s3.check_date
        )
    )
    ) AS streak_days
</select>

<!-- =====
Skill Distribution: total minutes per skill
===== -->

<select id="getSkillDistribution"
resultType="com.yourname.tracker.model.SkillStat">
    SELECT
        s.name      AS skillName,
        s.color_hex AS color,
        SUM(ls.minutes_practiced) AS totalMinutes
    FROM log_skill ls
    JOIN skill s ON ls.skill_id = s.id
    JOIN daily_log dl ON ls.daily_log_id = dl.id
    WHERE dl.user_id = #{userId}
        AND dl.log_date >= #{sinceDate}
    GROUP BY s.id, s.name, s.color_hex
    ORDER BY totalMinutes DESC
    LIMIT 10
</select>

<!-- =====
Weekly Hours: total hours grouped by ISO week
===== -->

<select id="getWeeklyHours"
resultType="com.yourname.tracker.model.WeeklyHours">
    SELECT

```

```

        CONCAT(YEAR(log_date), '-W', LPAD(WEEK(log_date, 3), 2, '0')) AS
weekLabel,
        SUM(hours_coded) AS totalHours
    FROM daily_log
    WHERE user_id = #{userId}
        AND log_date >= #{sinceDate}
    GROUP BY YEAR(log_date), WEEK(log_date, 3)
    ORDER BY YEAR(log_date) ASC, WEEK(log_date, 3) ASC
</select>

<!-- =====
        Total Days Logged
        ===== -->
<select id="getTotalDaysLogged" resultType="java.lang.Integer">
    SELECT COUNT(*) FROM daily_log WHERE user_id = #{userId}
</select>

</mapper>

```

Part 4: Service & Controller Layer (Day 3)

Step 4.1 — Create the Analytics Service

Create `src/main/java/com/yourname/tracker/service/AnalyticsService.java`.
This service aggregates data from the mapper and prepares it for the controllers.

```
package com.yourname.tracker.service;

import com.yourname.tracker.mapper.AnalyticsMapper;
import com.yourname.tracker.model.ActivityPoint;
import com.yourname.tracker.model.SkillStat;
import com.yourname.tracker.model.WeeklyHours;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

import java.time.LocalDate;
import java.util.List;

@Service
@RequiredArgsConstructor
public class AnalyticsService {

    private final AnalyticsMapper analyticsMapper;

    public List<ActivityPoint> getYearlyActivity(Long userId) {
        LocalDate end = LocalDate.now();
        LocalDate start = end.minusYears(1);
        return analyticsMapper.getDailyActivity(userId, start, end);
    }

    public int getCurrentStreak(Long userId) {
        Integer streak = analyticsMapper.getCurrentStreak(userId,
LocalDate.now());
        return streak != null ? streak : 0;
    }

    public List<SkillStat> getSkillDistributionLast30Days(Long userId) {
        return analyticsMapper.getSkillDistribution(userId,
LocalDate.now().minusDays(30));
    }

    public List<WeeklyHours> getWeeklyHoursLast12Weeks(Long userId) {
        return analyticsMapper.getWeeklyHours(userId,
LocalDate.now().minusWeeks(12));
    }

    public int getTotalDaysLogged(Long userId) {
        Integer total = analyticsMapper.getTotalDaysLogged(userId);
        return total != null ? total : 0;
    }
}
```



```
}  
}
```

Step 4.2 — Create the Dashboard Controller

Create `src/main/java/com/yourname/tracker/web/DashboardController.java`. For the MVP, we will use a hardcoded `userId` of `1L` (assuming a single user). You can add proper authentication later.

```
package com.yourname.tracker.web;  
  
import com.yourname.tracker.service.AnalyticsService;  
import lombok.RequiredArgsConstructor;  
import org.springframework.stereotype.Controller;  
import org.springframework.ui.Model;  
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.RequestMapping;  
  
@Controller  
@RequestMapping("/dashboard")  
@RequiredArgsConstructor  
public class DashboardController {  
  
    private final AnalyticsService analyticsService;  
    private static final Long MVP_USER_ID = 1L; // Replace with auth later  
  
    @GetMapping  
    public String dashboard(Model model) {  
        model.addAttribute("currentStreak",  
analyticsService.getCurrentStreak(MVP_USER_ID));  
        model.addAttribute("totalDays",  
analyticsService.getTotalDaysLogged(MVP_USER_ID));  
        model.addAttribute("skillStats",  
analyticsService.getSkillDistributionLast30Days(MVP_USER_ID));  
        model.addAttribute("weeklyHours",  
analyticsService.getWeeklyHoursLast12Weeks(MVP_USER_ID));  
        return "dashboard";  
    }  
}
```

Step 4.3 — Create the Heatmap REST API Endpoint

Cal-Heatmap fetches data via a REST call. Create `src/main/java/com/yourname/tracker/web/api/ActivityApiController.java`:

```
package com.yourname.tracker.web.api;

import com.yourname.tracker.mapper.AnalyticsMapper;
import com.yourname.tracker.model.ActivityPoint;
import lombok.RequiredArgsConstructor;
import org.springframework.format.annotation.DateTimeFormat;
import org.springframework.web.bind.annotation.*;

import java.time.LocalDate;
import java.util.List;

@RestController
@RequestMapping("/api/activity")
@RequiredArgsConstructor
public class ActivityApiController {

    private final AnalyticsMapper analyticsMapper;
    private static final Long MVP_USER_ID = 1L;

    @GetMapping
    public List<ActivityPoint> getActivity(
        @RequestParam @DateTimeFormat(iso = DateTimeFormat.ISO.DATE)
        LocalDate start,
        @RequestParam @DateTimeFormat(iso = DateTimeFormat.ISO.DATE)
        LocalDate end) {
        return analyticsMapper.getDailyActivity(MVP_USER_ID, start, end);
    }
}
```

Part 5: Frontend — The Dashboard Template (Day 4)

Step 5.1 — Create a Base Layout Fragment

Create `src/main/resources/templates/fragments/layout.html` for a reusable page structure:

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      th:fragment="layout(title, content)">
<head>
  <meta charset="UTF-8"/>
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title th:replace="${title}">Dev Habit Tracker</title>
  <!-- Bootstrap 5 -->
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
  <!-- Cal-Heatmap -->
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/cal-
heatmap@4.2.2/dist/cal-heatmap.css"/>
  <style>
    body { background-color: #f8f9fa; }
    .stat-card { border-left: 4px solid #0d6efd; }
    .streak-badge { font-size: 2rem; font-weight: bold; color: #fd7e14;
  }
</style>
</head>
<body>
<nav class="navbar navbar-dark bg-dark mb-4">
  <div class="container">
    <a class="navbar-brand" href="/dashboard">☰ Dev Habit Tracker</a>
    <div>
      <a class="btn btn-outline-light btn-sm me-2" href="/daily-
logs/add">+ Log Today</a>
      <a class="btn btn-outline-light btn-sm" href="/skills">Manage
Skills</a>
    </div>
  </div>
</nav>
<div class="container" th:replace="${content}">
  <!-- Page content here -->
</div>
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min
</script>
<script src="https://d3js.org/d3.v7.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/cal-heatmap@4.2.2/dist/cal-
heatmap.min.js"></script>
<script
src="https://cdn.jsdelivr.net/npm/chart.js@4.4.0/dist/chart.umd.min.js">
</script>

```

```
</body>  
</html>
```

Step 5.2 — Create the Main Dashboard Template

Create `src/main/resources/templates/dashboard.html`:

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      th:replace="~{fragments/layout :: layout(~{::title}, ~{::section})}">
<head>
  <title>Dashboard | Dev Habit Tracker</title>
</head>
<body>
<section>
  <!-- ===== STAT CARDS ===== -->
  <div class="row g-3 mb-4">
    <div class="col-md-4">
      <div class="card stat-card shadow-sm h-100">
        <div class="card-body">
          <h6 class="text-muted">Current Streak</h6>
          <div class="streak-badge" th:text="${currentStreak} + '
days 🏆'">0 days 🏆</div>
        </div>
      </div>
    </div>
    <div class="col-md-4">
      <div class="card stat-card shadow-sm h-100">
        <div class="card-body">
          <h6 class="text-muted">Total Days Logged</h6>
          <div class="streak-badge" th:text="${totalDays}">0</div>
        </div>
      </div>
    </div>
    <div class="col-md-4">
      <div class="card stat-card shadow-sm h-100">
        <div class="card-body">
          <h6 class="text-muted">Today's Status</h6>
          <a href="/daily-logs/add" class="btn btn-primary mt-
2">Log Today's Session</a>
        </div>
      </div>
    </div>
  </div>

  <!-- ===== CONTRIBUTION HEATMAP ===== -->
  <div class="card shadow-sm mb-4">
    <div class="card-body">
      <h5 class="card-title">Contribution Activity (Last 12 Months)
</h5>
      <div id="cal-heatmap" class="overflow-auto py-2"></div>
    </div>
  </div>

```

```

</div>

<!-- ===== CHARTS ROW ===== -->
<div class="row g-3">
  <div class="col-md-5">
    <div class="card shadow-sm h-100">
      <div class="card-body">
        <h5 class="card-title">Skills (Last 30 Days)</h5>
        <canvas id="skillsPieChart"></canvas>
      </div>
    </div>
  </div>
  <div class="col-md-7">
    <div class="card shadow-sm h-100">
      <div class="card-body">
        <h5 class="card-title">Weekly Hours (Last 12 Weeks)</h5>
        <canvas id="weeklyBarChart"></canvas>
      </div>
    </div>
  </div>
</div>

<!-- ===== JAVASCRIPT FOR CHARTS ===== -->
<script th:inline="javascript">
  // --- 1. Cal-Heatmap (Contribution Calendar) ---
  const cal = new CalHeatmap();
  cal.paint({
    itemSelector: "#cal-heatmap",
    domain: { type: 'month', label: { text: 'MMM', position: 'top' }
  },
    subDomain: { type: 'day', radius: 2, width: 14, height: 14,
gutter: 2 },
    date: { start: new Date(new Date().setFullYear(new
Date().getFullYear() - 1)) },
    range: 13,
    data: {
      source: '/api/activity?start={{start=YYYY-MM-DD}}&end=
{{end=YYYY-MM-DD}}',
      x: 'timestamp',
      y: (d) => +d['value'],
      groupY: 'sum'
    },
    scale: {
      color: {
        type: 'threshold',
        range: ['#ebedf0', '#9be9a8', '#40c463', '#30a14e',

```

```

'#216e39'],
    domain: [0, 2, 4, 6, 8]
  }
}
});

// --- 2. Skills Pie Chart (Chart.js) ---
const skillStats = /*[[$ {skillStats}]]*/ [];
if (skillStats.length > 0) {
  new Chart(document.getElementById('skillsPieChart'), {
    type: 'doughnut',
    data: {
      labels: skillStats.map(s => s.skillName),
      datasets: [{
        data: skillStats.map(s => s.totalMinutes),
        backgroundColor: skillStats.map(s => s.color ||
'#6c757d'),
        hoverOffset: 4
      }]
    },
    options: {
      plugins: {
        legend: { position: 'right' },
        tooltip: {
          callbacks: {
            label: (ctx) => ` ${Math.round(ctx.raw / 60
* 10) / 10} hrs`
          }
        }
      }
    }
  });
}

// --- 3. Weekly Hours Bar Chart (Chart.js) ---
const weeklyData = /*[[$ {weeklyHours}]]*/ [];
if (weeklyData.length > 0) {
  new Chart(document.getElementById('weeklyBarChart'), {
    type: 'bar',
    data: {
      labels: weeklyData.map(w => w.weekLabel),
      datasets: [{
        label: 'Hours Coded',
        data: weeklyData.map(w => w.totalHours),
        backgroundColor: 'rgba(13, 110, 253, 0.7)',
        borderColor: 'rgba(13, 110, 253, 1)',

```

```

        borderWidth: 1,
        borderRadius: 4
    }]
},
options: {
    scales: {
        y: { beginAtZero: true, title: { display: true,
text: 'Hours' } }
    },
    plugins: { legend: { display: false } }
}
});
}
</script>
</section>
</body>
</html>

```

Step 5.3 — Create the Daily Log Entry Form

Create `src/main/resources/templates/dailyLog/add.html`. This is the form users fill in each day.


```

Practiced</label>
<div th:each="skill : ${allSkills}" class="form-
check">
    <input class="form-check-input"
type="checkbox"
th:name="'skill_' + ${skill.id}"
th:id="'skill_' + ${skill.id}"/>
    <label class="form-check-label"
th:for="'skill_' + ${skill.id}"
th:text="${skill.name}">Skill</label>
</div>
</div>
<div class="mb-3">
    <label class="form-label fw-bold">Notes</label>
    <textarea name="notes" class="form-control"
rows="3"
placeholder="What did you work on
today?"></textarea>
</div>
<div class="d-flex gap-2">
    <button type="submit" class="btn btn-
primary">Save Log</button>
    <a href="/dashboard" class="btn btn-outline-
secondary">Cancel</a>
</div>
</form>
</div>
</div>
</div>
</section>
</body>
</html>

```

Part 6: Seed Data (Day 4, continued)

To test the dashboard, add some initial data. Create a `DataInitializer` component that runs on startup.

Create `src/main/java/com/yourname/tracker/config/DataInitializer.java`:

```
package com.yourname.tracker.config;

import com.yourname.tracker.domain.Skill;
import com.yourname.tracker.repos.SkillRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

import java.util.List;

@Component
@RequiredArgsConstructor
public class DataInitializer implements CommandLineRunner {

    private final SkillRepository skillRepository;

    @Override
    public void run(String... args) {
        if (skillRepository.count() == 0) {
            skillRepository.saveAll(List.of(
                createSkill("Java", "Language", "#b07219"),
                createSkill("Spring Boot", "Framework", "#6db33f"),
                createSkill("MySQL", "Database", "#00758f"),
                createSkill("React", "Frontend", "#61dafb"),
                createSkill("Docker", "DevOps", "#2496ed"),
                createSkill("MyBatis", "Framework", "#dc382d"),
                createSkill("Git", "Tool", "#f05032")
            ));
        }
    }

    private Skill createSkill(String name, String category, String color) {
        Skill s = new Skill();
        s.setName(name);
        s.setCategory(category);
        s.setColorHex(color);
        return s;
    }
}
```

Part 7: Deployment to Railway (Day 5)

Step 7.1 — Create Production Configuration

Create `src/main/resources/application-prod.properties`:

```
# Production database (Railway injects these as environment variables)
spring.datasource.url=${MYSQL_URL}
spring.datasource.username=${MYSQL_USER}
spring.datasource.password=${MYSQL_PASSWORD}
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false

# Disable Swagger in production (optional)
springdoc.api-docs.enabled=false
springdoc.swagger-ui.enabled=false
```

Step 7.2 — Create the Dockerfile

Create a `Dockerfile` in the project root. This multi-stage build produces a lean final image.

```
# ---- Build Stage ----
FROM maven:3.9.6-eclipse-temurin-21-alpine AS build
WORKDIR /app
COPY pom.xml .
# Download dependencies first (cached layer)
RUN mvn dependency:go-offline -B
COPY src ./src
RUN mvn clean package -DskipTests -B

# ---- Run Stage ----
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-Xmx256m", "-jar", "app.jar"]
```

Step 7.3 — Add a `.dockerignore` File

Create `.dockerignore` to keep the build context small:

```
target/  
.git/  
*.md  
.mvn/
```

Step 7.4 — Push to GitHub

```
git add .  
git commit -m "feat: complete MVP with dashboard, MyBatis analytics, and  
Dockerfile"  
git remote add origin https://github.com/YOUR_USERNAME/dev-habit-tracker.git  
git push -u origin main
```

Step 7.5 — Deploy on Railway

Follow these steps on railway.app:

1. **Create a new project:** Click **New Project** → **Deploy from GitHub repo** → select your `dev-habit-tracker` repository.
2. **Add a MySQL database:** In the same project, click **+ New** → **Database** → **Add MySQL**. Railway provisions a managed MySQL instance automatically.
3. **Configure environment variables:** In your Spring Boot service, go to the **Variables** tab. Railway automatically exposes the MySQL connection variables. You need to add:
 - `SPRING_PROFILES_ACTIVE = prod`
 - `MYSQL_URL = ${MYSQLHOST}:${MYSQLPORT}/${MYSQLDATABASE}` (or use the `MYSQL_URL` variable Railway provides directly)
 - `MYSQL_USER = ${MYSQLUSER}`
 - `MYSQL_PASSWORD = ${MYSQLPASSWORD}`

4. **Trigger a deployment:** Railway will detect the `Dockerfile`, build the image, and deploy. Monitor the build logs in the **Deployments** tab.
5. **Generate a public URL:** Go to **Settings** → **Networking** → **Generate Domain**. Your application is now live.

Railway Cost Estimate

Railway's pricing for this MVP is very manageable. The Spring Boot service and MySQL together will consume approximately \$3–7 per month, depending on traffic and uptime. You can set a spending limit in the Railway dashboard to prevent unexpected charges.

Part 8: Project File Structure Summary

After completing all steps, your project structure will look like this:

```

dev-habit-tracker/
├─ src/
│   └─ main/
│       ├── java/com/yourname/tracker/
│       │   ├── config/
│       │   │   └─ DataInitializer.java      # Seeds initial skill data
│       │   └─ domain/                      # JPA Entities (generated by
│       │       Bootify)
│       │       ├── User.java
│       │       ├── Skill.java
│       │       ├── DailyLog.java
│       │       │   └─ LogSkill.java
│       │       └─ mapper/
│       │           ├── AnalyticsMapper.java  # MyBatis mapper interface
│       │           └─ model/                 # MyBatis result POJOs
│       │               ├── ActivityPoint.java
│       │               ├── SkillStat.java
│       │               └─ WeeklyHours.java
│       └─ repos/                           # Spring Data JPA repos
│           (generated)
│           └─ rest/                         # REST controllers
│               (generated by Bootify)
│               ├── service/
│               │   └─ AnalyticsService.java  # Analytics business logic
│               └─ web/
│                   ├── DashboardController.java # Thymeleaf dashboard
│                   └─ api/
│                       └─ ActivityApiController.java # REST for heatmap
│               └─ resources/
│                   ├── mapper/
│                   │   └─ AnalyticsMapper.xml # MyBatis SQL queries
│                   └─ templates/
│                       ├── fragments/
│                       │   └─ layout.html      # Base layout
│                       ├── dashboard.html      # Main dashboard
│                       └─ dailyLog/
│                           └─ add.html         # Log entry form
│                   ├── application.properties # Local config
│                   └─ application-prod.properties # Production config
├─ Dockerfile
├─ .dockerignore
└─ pom.xml

```

Part 9: Future Enhancements

This MVP is a strong foundation. The following features are natural next steps:

Enhancement	Description	Effort
User Authentication	Add Spring Security with form-based login and registration.	Medium
Goal Setting	Implement the <code>GOAL</code> entity; allow users to set weekly/monthly targets.	Medium
Log History Page	A paginated table of all past log entries with edit/delete actions.	Low
Focus Score Trend	A line chart tracking focus score over time.	Low
Longest Streak	A MyBatis query to find the all-time longest streak.	Low
Email Reminders	A scheduled job to email the user if they haven't logged by 9 PM.	Medium
GitHub Integration	Optionally sync with the GitHub API to auto-populate commit activity.	High

References

1. [Bootify — Free Plan Quickstart 1](#)
2. [MyBatis Spring Boot Starter Documentation 2](#)
3. [Cal-Heatmap v4 Documentation — Data Options 3](#)
4. [Chart.js Documentation 4](#)
5. [Railway — Deploy a Spring Boot App 5](#)
6. [MySQL Recursive CTE for Streak Calculation 6](#)